

SOC Analyst Lab

Introduction

This project involves analyzing log data to determine what happened to a compromised web server honeypot. Essentially, it is a Capture the Flag (CTF) challenge. The files from the compromised web server were sourced from cyberdefenders.org¹ under the lab titled "Hammered Lab."

Objectives

Here is a summary of the objectives for this lab

- Demonstrating the ability to deploy virtual machines and install operating systems
- Showcasing the use of Linux and command line.
- Incident response and detection
- Log analysis
- Threat Detection

Setup

In order to analyze the logs, a lab is required to be built. This will involve creating a virtual machine using VirtualBox, running Kali Linux on macOS platform.

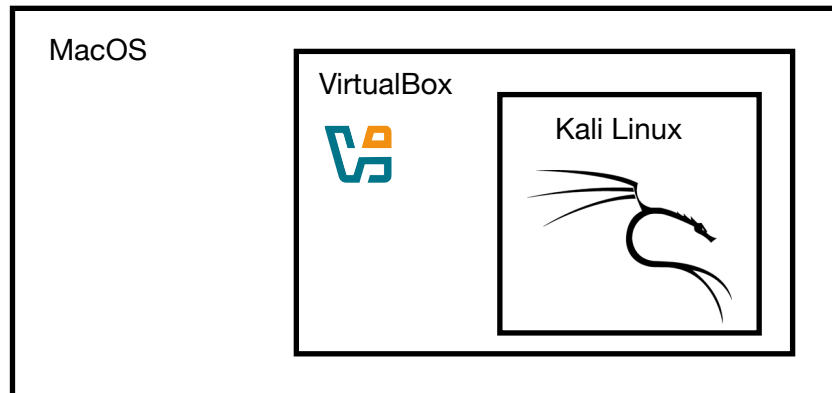


Figure 1: Visual Representation of the lab

¹ <https://cyberdefenders.org/blueteam-ctf-challenges/hammered/>

The software that will be used in this lab

- VirtualBox 7.1.4
- Kali 2024.4
- macOS Sequoia

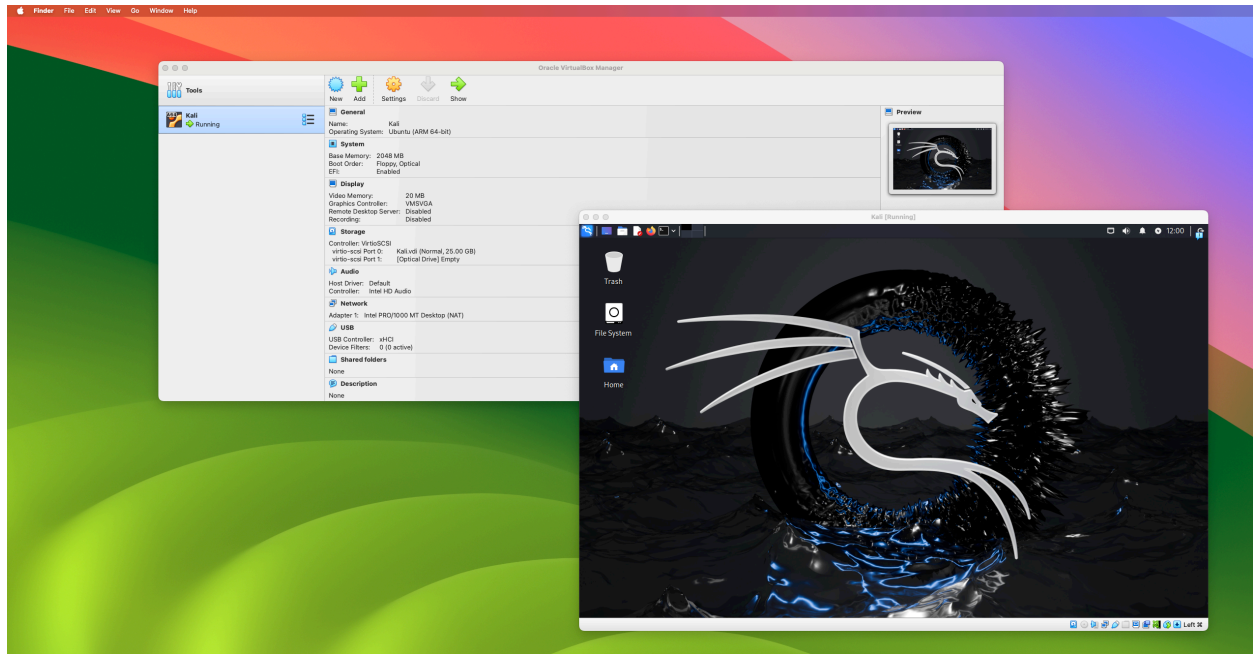


Figure 2: Screenshot of the environment

Next step is to download and extract the files from cyberdefenders.org.

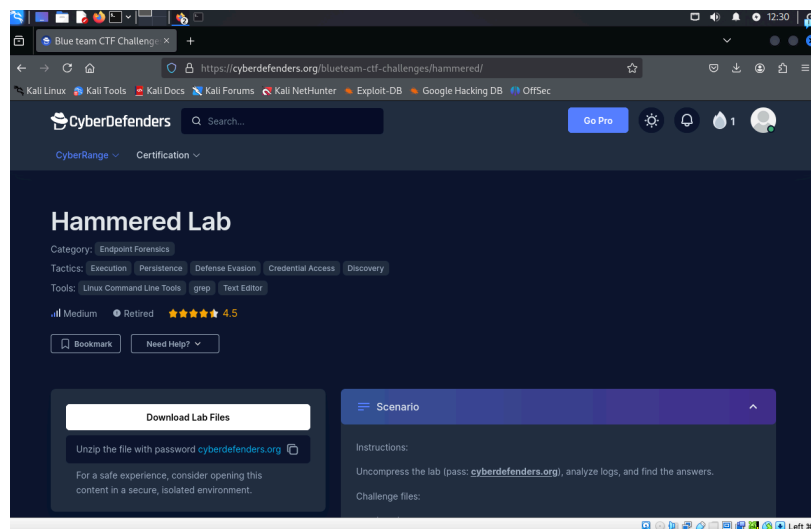
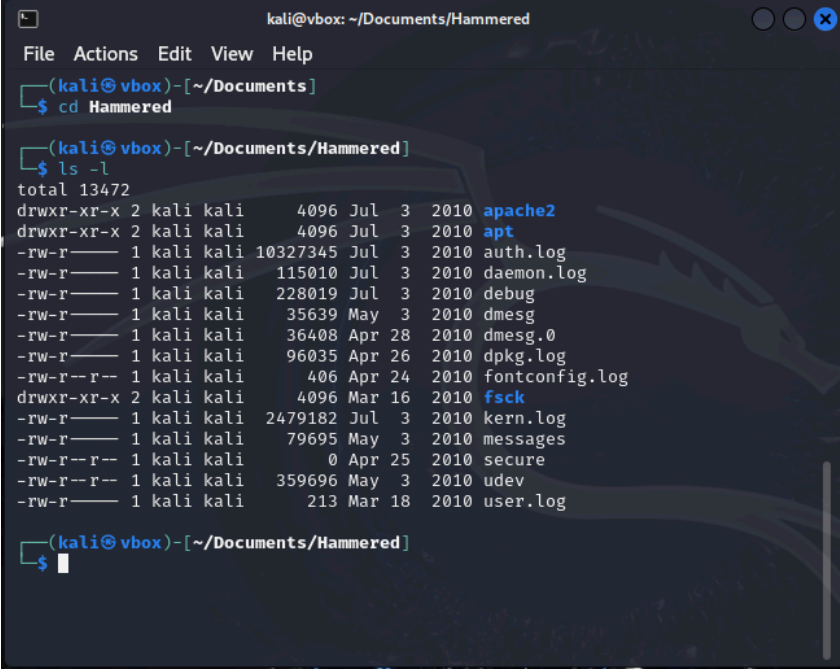


Figure 3: Cyberdefenders website

Using the terminal can confirm that the files have been downloaded and extracted.

A terminal window titled 'kali@vbox: ~/Documents/Hammered' with a menu bar (File, Actions, Edit, View, Help). The prompt is '(kali@vbox)-[~/Documents/Hammered]'. The command '\$ cd Hammered' has been executed. The next command '\$ ls -l' has been executed, resulting in a long listing of files and directories. The output shows permissions, owner, group, size, date, and filename for each item. The files listed are: apache2, apt, auth.log, daemon.log, debug, dmesg, dmesg.0, dpkg.log, fontconfig.log, fsck, kern.log, messages, secure, udev, and user.log.

```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)-[~/Documents]
$ cd Hammered

(kali@vbox)-[~/Documents/Hammered]
$ ls -l
total 13472
drwxr-xr-x 2 kali kali 4096 Jul 3 2010 apache2
drwxr-xr-x 2 kali kali 4096 Jul 3 2010 apt
-rw-r--r-- 1 kali kali 10327345 Jul 3 2010 auth.log
-rw-r--r-- 1 kali kali 115010 Jul 3 2010 daemon.log
-rw-r--r-- 1 kali kali 228019 Jul 3 2010 debug
-rw-r--r-- 1 kali kali 35639 May 3 2010 dmesg
-rw-r--r-- 1 kali kali 36408 Apr 28 2010 dmesg.0
-rw-r--r-- 1 kali kali 96035 Apr 26 2010 dpkg.log
-rw-r--r-- 1 kali kali 406 Apr 24 2010 fontconfig.log
drwxr-xr-x 2 kali kali 4096 Mar 16 2010 fsck
-rw-r--r-- 1 kali kali 2479182 Jul 3 2010 kern.log
-rw-r--r-- 1 kali kali 79695 May 3 2010 messages
-rw-r--r-- 1 kali kali 0 Apr 25 2010 secure
-rw-r--r-- 1 kali kali 359696 May 3 2010 udev
-rw-r--r-- 1 kali kali 213 Mar 18 2010 user.log

(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 4: List of files for the lab

The setup is complete and now to begin the investigation.

Investigation

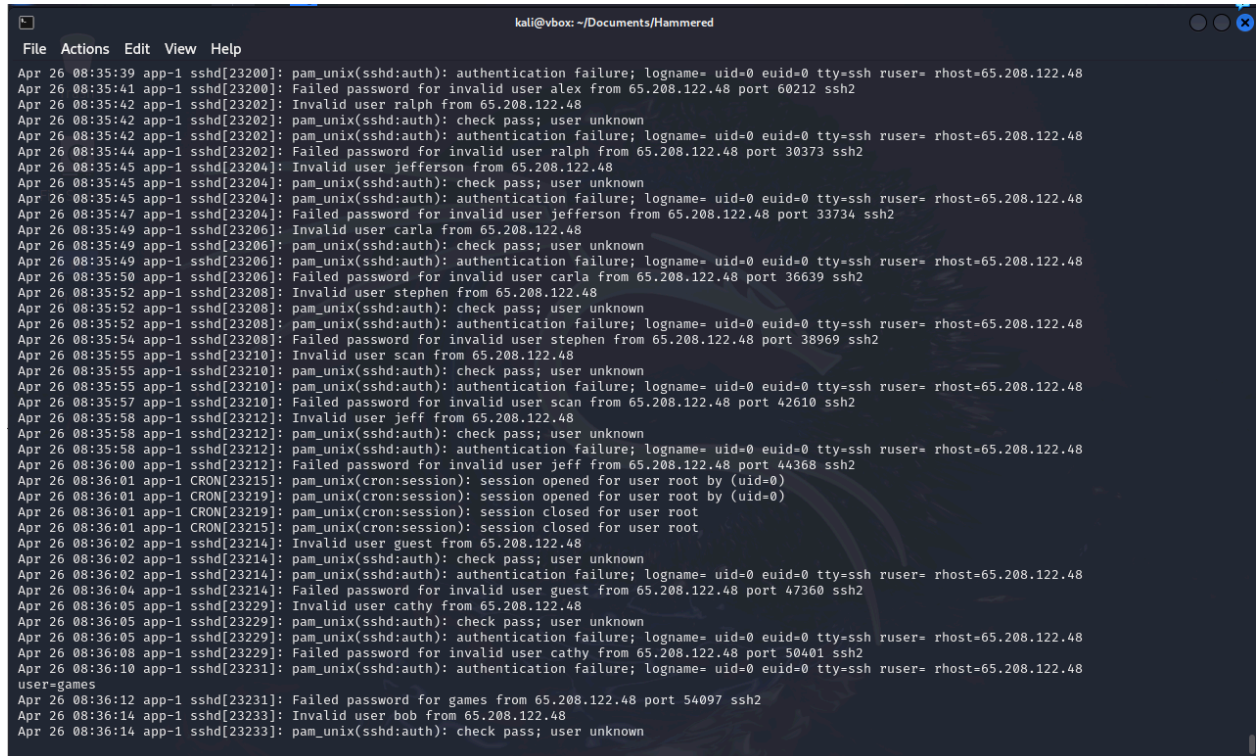
To begin the investigation, I will use the website challenge questions as a guide to help discovering whether or not the honeypot server has been compromised. To start with though, looking through the files in the Hammered directory, there are 5 files of interest.

- kern.log - Contain system messages from the Linux kernel
- auth.log - Keeps track of authentication and security related events
- daemon.log - Records system service and background process events
- dmesg - Displays kernel ring buffer messages and hardware events
- apache2 - Record web server access and error details

What service did the attackers use to gain access to the system?

We can refer to the auth.log to check for any failed or successful login attempts. This should show what the attacker was trying to log into.

The screenshot below shows a snapshot of the auth.log. There are multiple authentication failures using the SSH service.



```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
Apr 26 08:35:39 app-1 sshd[23200]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:35:41 app-1 sshd[23200]: Failed password for invalid user alex from 65.208.122.48 port 60212 ssh2
Apr 26 08:35:42 app-1 sshd[23202]: Invalid user ralph from 65.208.122.48
Apr 26 08:35:42 app-1 sshd[23202]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:35:42 app-1 sshd[23202]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:35:44 app-1 sshd[23202]: Failed password for invalid user ralph from 65.208.122.48 port 30373 ssh2
Apr 26 08:35:45 app-1 sshd[23204]: Invalid user jefferson from 65.208.122.48
Apr 26 08:35:45 app-1 sshd[23204]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:35:45 app-1 sshd[23204]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:35:47 app-1 sshd[23204]: Failed password for invalid user jefferson from 65.208.122.48 port 33734 ssh2
Apr 26 08:35:49 app-1 sshd[23206]: Invalid user carla from 65.208.122.48
Apr 26 08:35:49 app-1 sshd[23206]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:35:49 app-1 sshd[23206]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:35:50 app-1 sshd[23206]: Failed password for invalid user carla from 65.208.122.48 port 36639 ssh2
Apr 26 08:35:52 app-1 sshd[23208]: Invalid user stephen from 65.208.122.48
Apr 26 08:35:52 app-1 sshd[23208]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:35:52 app-1 sshd[23208]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:35:54 app-1 sshd[23208]: Failed password for invalid user stephen from 65.208.122.48 port 38969 ssh2
Apr 26 08:35:55 app-1 sshd[23210]: Invalid user scan from 65.208.122.48
Apr 26 08:35:55 app-1 sshd[23210]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:35:55 app-1 sshd[23210]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:35:57 app-1 sshd[23210]: Failed password for invalid user scan from 65.208.122.48 port 42610 ssh2
Apr 26 08:35:58 app-1 sshd[23212]: Invalid user jeff from 65.208.122.48
Apr 26 08:35:58 app-1 sshd[23212]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:35:58 app-1 sshd[23212]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:36:00 app-1 sshd[23212]: Failed password for invalid user jeff from 65.208.122.48 port 44368 ssh2
Apr 26 08:36:01 app-1 CRON[23215]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 26 08:36:01 app-1 CRON[23219]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 26 08:36:01 app-1 CRON[23219]: pam_unix(cron:session): session closed for user root
Apr 26 08:36:02 app-1 sshd[23214]: Invalid user guest from 65.208.122.48
Apr 26 08:36:02 app-1 sshd[23214]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:36:02 app-1 sshd[23214]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:36:04 app-1 sshd[23214]: Failed password for invalid user guest from 65.208.122.48 port 47360 ssh2
Apr 26 08:36:05 app-1 sshd[23229]: Invalid user cathy from 65.208.122.48
Apr 26 08:36:05 app-1 sshd[23229]: pam_unix(sshd:auth): check pass; user unknown
Apr 26 08:36:05 app-1 sshd[23229]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:36:08 app-1 sshd[23229]: Failed password for invalid user cathy from 65.208.122.48 port 50401 ssh2
Apr 26 08:36:10 app-1 sshd[23231]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
user=games
Apr 26 08:36:12 app-1 sshd[23231]: Failed password for games from 65.208.122.48 port 54097 ssh2
Apr 26 08:36:14 app-1 sshd[23233]: Invalid user bob from 65.208.122.48
Apr 26 08:36:14 app-1 sshd[23233]: pam_unix(sshd:auth): check pass; user unknown
```

Figure 5: Snapshot of auth.log

The sheer number of failed logins, in a relatively short space of time is an indicator of comprise for a brute force attempt.

The service being attacked is SSH.

What is the operating system version of the targeted system?

The dmesg logs contains kernel ring buffer data. This data can provide insights into the system startup, hardware failures and kernel issues. In this case, we can output the beginning of the file to show what operating system it is running.

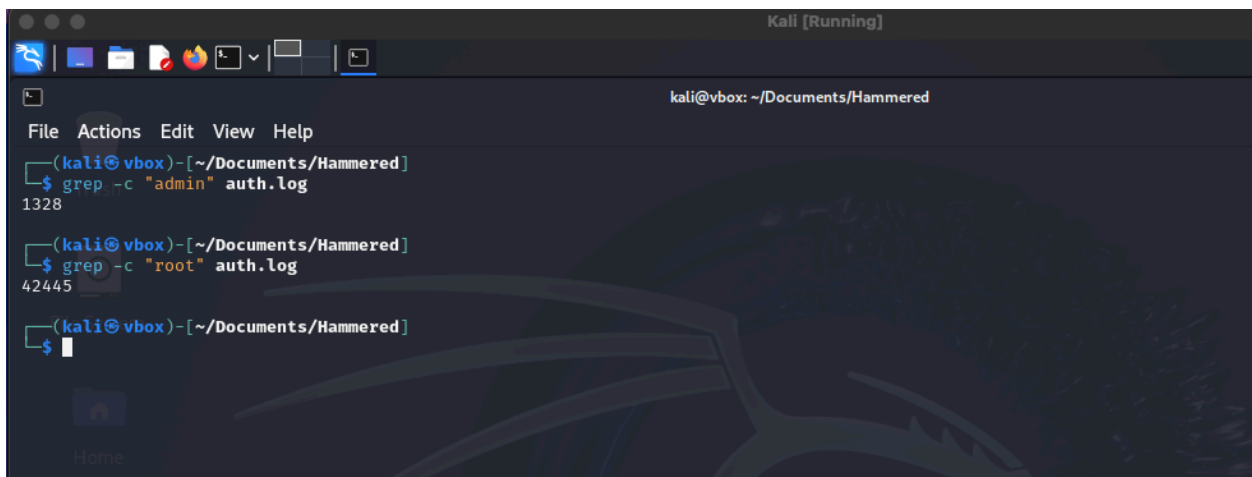
```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)~-[~/Documents/Hammered]
$ cat dmesg | head
[ 0.000000] Initializing cgroup subsys cpuset
[ 0.000000] Initializing cgroup subsys cpu
[ 0.000000] Linux version 2.6.24-26-server (buildd@crested) (gcc version 4.2.4 (Ubuntu 4.2.4-1ubuntu3)) #1 SMP Tue Dec 1 18
-26.64-server)
[ 0.000000] Command line: root=UUID=a691743a-a4b7-482d-95ff-406e5acd83a3 ro quiet splash
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: 0000000000000000 - 00000000000009f800 (usable)
[ 0.000000] BIOS-e820: 00000000000009f800 - 0000000000000a0000 (reserved)
[ 0.000000] BIOS-e820: 0000000000000ca000 - 0000000000000cc000 (reserved)
[ 0.000000] BIOS-e820: 0000000000000dc000 - 0000000000000e4000 (reserved)
[ 0.000000] BIOS-e820: 000000000000e8000 - 00000000000100000 (reserved)
```

Figure 6: Output of first 10 lines of dmesg log

The operating system in question is: 4.2.4-1ubuntu3.

Q3: What is the name of the compromised account?

The auth.log used previously its likely to contain the answer. From an attackers perspective, they will seek to compromise the account with the highest privileges. My assumption is an administrative account of some form. Using the grep command can search for occurrences of the word 'admin' and 'root'. Both of these would indicate the attempts made into the system.



```
Kali [Running]
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)-[~/Documents/Hammered]
$ grep -c "admin" auth.log
1328
(kali@vbox)-[~/Documents/Hammered]
$ grep -c "root" auth.log
42445
(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 7: Using grep to count occurrences of a word

From the results of the grep command, the attacker was after the root account given the number of occurrences, 42445 vs 1328 for 'admin' in the auth.log file.

The name of the compromised account is 'root'.

Q4: How many attackers, represented by unique IP addresses, were able to successful access the system after initial failed attempts?

Using the auth.log again, we can run a grep command, looking for the string 'accepted password' and 'root', this would indicate a successful login. Can also use the awk command to retrieve the fields that are of interest. In this case, it will be the IP addresses.

The following command will be run.

```
cat auth.log | grep -i "accepted password" | grep root | awk '{print $11}' | uniq
```

```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help

(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "accepted password" | grep root | awk '{print $11}' | uniq
10.0.1.2
219.150.161.20
222.66.204.246
201.229.176.217
190.167.70.87
190.166.87.164
121.11.66.70
193.1.186.197
151.81.205.100
151.82.3.201
151.81.204.141
222.169.224.197
122.226.202.12
121.11.66.70
61.168.227.12
188.131.22.69
190.167.74.184
94.52.185.9
188.131.23.37

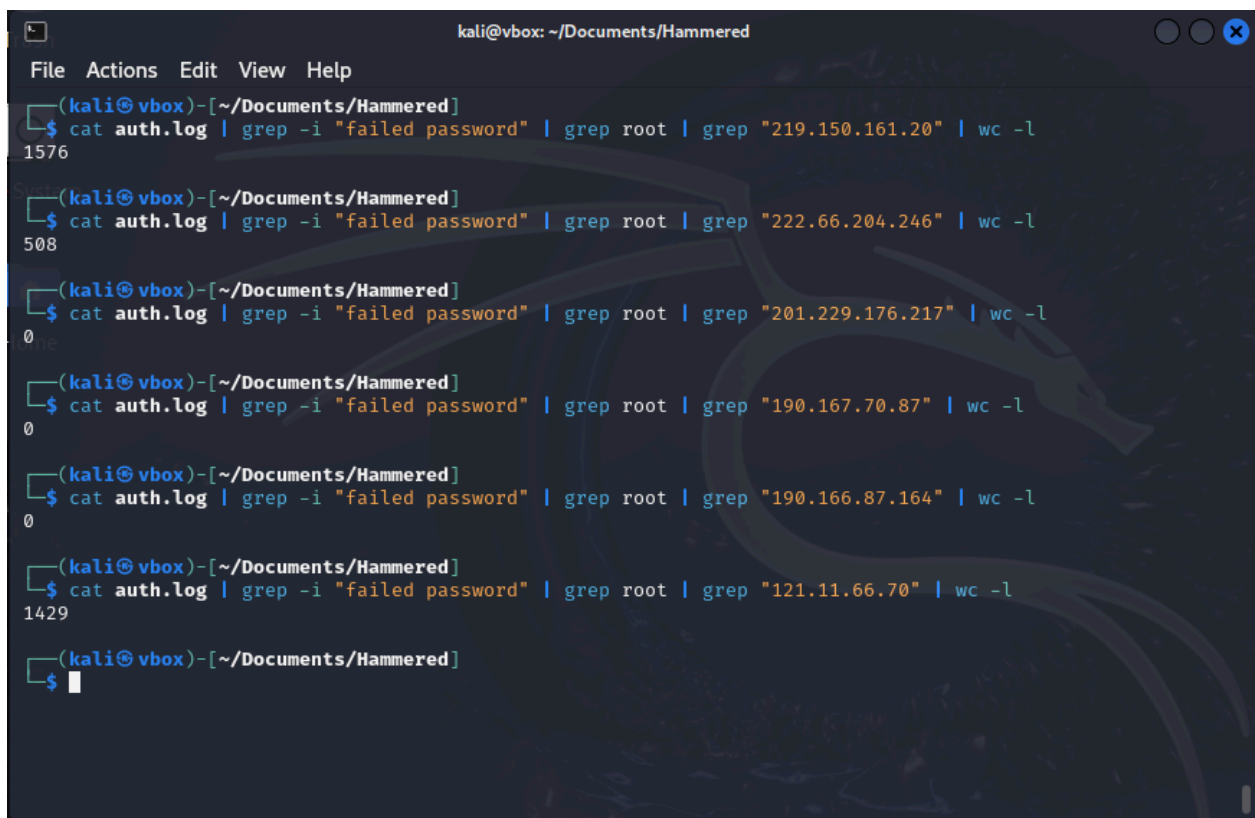
(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 8: Out of grep command

However, not all the IP addresses above are from the attacker. We can filter using 'failed password' term with the IP addresses from figure 8.

The `wc -l` in the command below will count the number of lines that match the grep patterns. In this case, its failed password, root and the IP addresses. A high number of matches for an IP address will indicate a brute force attempt and thus highly likely from the attacker.

```
cat auth.log | grep - "failed password" | grep root | grep "IP Address" | wc -l
```

```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "failed password" | grep root | grep "219.150.161.20" | wc -l
1576
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "failed password" | grep root | grep "222.66.204.246" | wc -l
508
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "failed password" | grep root | grep "201.229.176.217" | wc -l
0
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "failed password" | grep root | grep "190.167.70.87" | wc -l
0
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "failed password" | grep root | grep "190.166.87.164" | wc -l
0
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "failed password" | grep root | grep "121.11.66.70" | wc -l
1429
(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 9: Sample of each Command

The figure above is a snapshot of the command being run for the IP addresses. Not all IP addresses are shown. So far IP address 219.150.161.20, 222.66.204.246 and 121.11.66.70 are from the attacker given the high number of login attempts, ranging from 508 to 1576.

Referring back the figure 8, out of the 18 IP addresses, 6 have been identified as being from the attacker.

219.150.161.20
222.66.204.246
121.11.66.70
222.169.224.197
122.226.202.12
61.168.227.12

Figure 10: IP Addresses from Attacker

There were 6 attackers with unique IP Addresses

Q5: Which attackers IP address successfully logged into the system the most number of times?

In question 4, we identified the number of times each IP address tried to login to the system. The command that produced the highest count is the answer.

The IP address 219.150.161.20 is the attacker that successfully logged into the system.

Q6: How many requests were sent to the Apache Server

Turn our focus to the 'apache2' directory. The contents of the file 'www-access.log' will contain login entries. By counting the number of entries or lines in this file will determine how many requests were sent to the server. The following command will be used.

```
cat www-access.log | wc -l
```

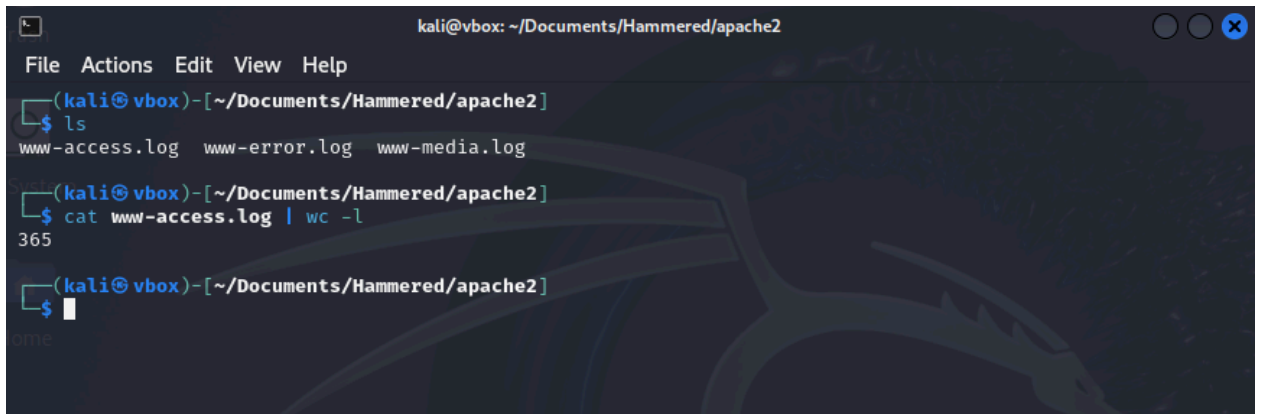
A screenshot of a terminal window titled 'kali@vbox: ~/Documents/Hammered/apache2'. The terminal shows a sequence of commands and their outputs. First, the user runs 'ls', which lists 'www-access.log', 'www-error.log', and 'www-media.log'. Then, the user runs 'cat www-access.log | wc -l', and the terminal outputs '365'. The terminal has a dark background with a faint dragon logo in the bottom right corner.

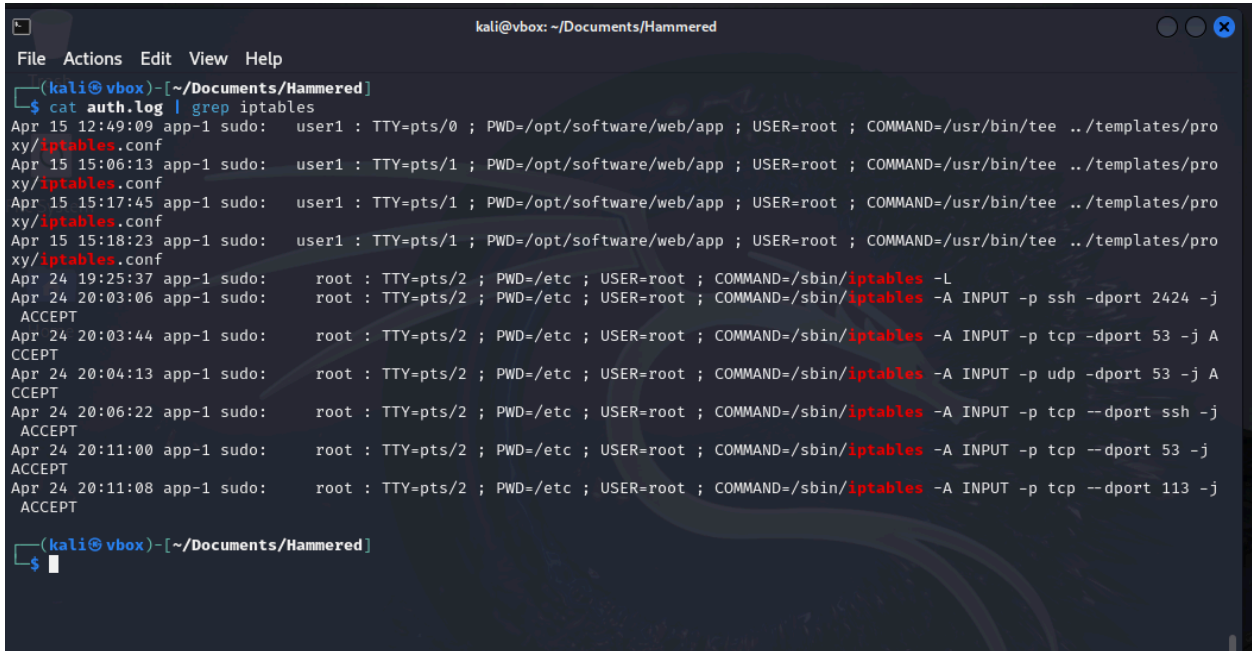
Figure 11: Output number of lines in www-access.log file

Therefore 365 requests were sent to the Apache Server.

Q7: How many rules have been added to the firewall?

Within Linux, the 'iptables' are used to manage firewall rules and network traffic. In this scenario, the iptables are located in the auth.log. Simply output the contents of the file with lines matching the pattern 'iptables'.

```
cat auth.log | grep iptables
```



```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)~-[~/Documents/Hammered]
$ cat auth.log | grep iptables
Apr 15 12:49:09 app-1 sudo: user1 : TTY=pts/0 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/pro
xy/iptables.conf
Apr 15 15:06:13 app-1 sudo: user1 : TTY=pts/1 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/pro
xy/iptables.conf
Apr 15 15:17:45 app-1 sudo: user1 : TTY=pts/1 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/pro
xy/iptables.conf
Apr 15 15:18:23 app-1 sudo: user1 : TTY=pts/1 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/pro
xy/iptables.conf
Apr 24 19:25:37 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -L
Apr 24 20:03:06 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p ssh -dport 2424 -j
ACCEPT
Apr 24 20:03:44 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp -dport 53 -j A
CCEPT
Apr 24 20:04:13 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p udp -dport 53 -j A
CCEPT
Apr 24 20:06:22 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport ssh -j
ACCEPT
Apr 24 20:11:00 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport 53 -j
ACCEPT
Apr 24 20:11:08 app-1 sudo: root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport 113 -j
ACCEPT
(kali@vbox)~-[~/Documents/Hammered]
$
```

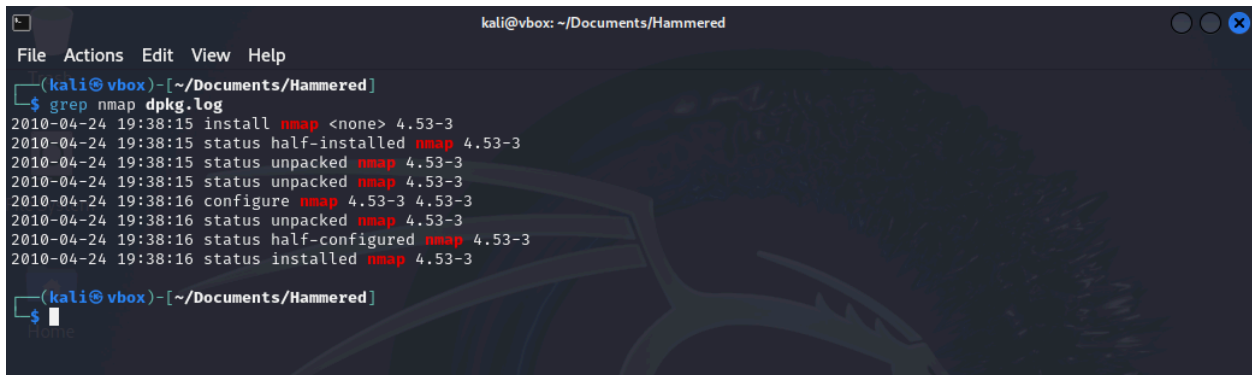
Figure 12: Output of auth.log containing iptables

From the output above shows firewall rules. Any line that has '-A INPUT...' signifies that its a rule that allows traffic to a destination port.

There are 6 firewall rules.

Q8: One of the downloaded files to the target system is a scanning tool. Provide the tool name

The dpkg.log is a log file that contains records of packages (programs) installed, removed, upgraded and configured on a Debian Linux system. This log file is likely to contain an entry for a scanning tool. After running the cat command on the dpkg.log and browsing through the output, the tool nmap came up. To narrow it down, I used grep with the pattern nmap to confirm my findings.

A terminal window titled 'kali@vbox: ~/Documents/Hammered' showing the command 'grep nmap dpkg.log' and its output. The output lists several log entries for the installation of nmap version 4.53-3 on 2010-04-24 at 19:38:15. The entries show the package being installed, then half-installed, then unpacked, then configured, then half-configured, and finally installed.

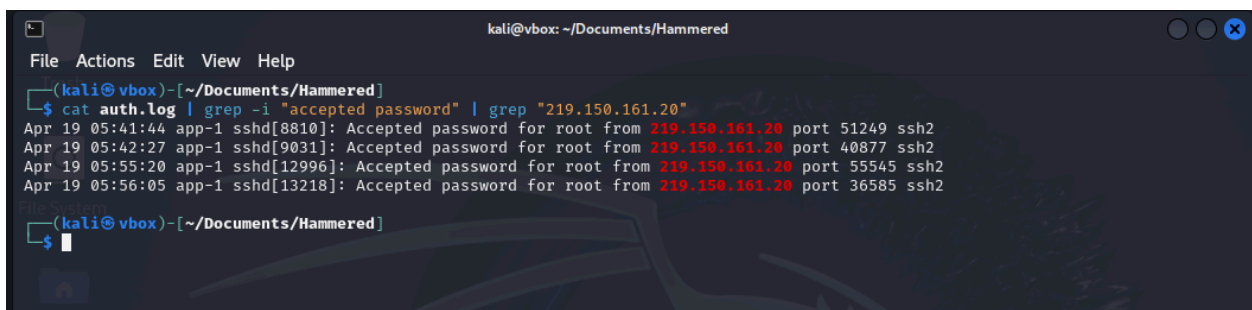
```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)-[~/Documents/Hammered]
$ grep nmap dpkg.log
2010-04-24 19:38:15 install nmap <none> 4.53-3
2010-04-24 19:38:15 status half-installed nmap 4.53-3
2010-04-24 19:38:15 status unpacked nmap 4.53-3
2010-04-24 19:38:15 status unpacked nmap 4.53-3
2010-04-24 19:38:16 configure nmap 4.53-3 4.53-3
2010-04-24 19:38:16 status unpacked nmap 4.53-3
2010-04-24 19:38:16 status half-configured nmap 4.53-3
2010-04-24 19:38:16 status installed nmap 4.53-3
(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 13: nmap entry in dpkg.log

Q9: When was the last login from the attacker with IP 219.150.161.20? Format: MM/DD/YYYY HH:MM:SS AM

Querying the auth.log and displaying lines that have successfully logged in with the IP address above.

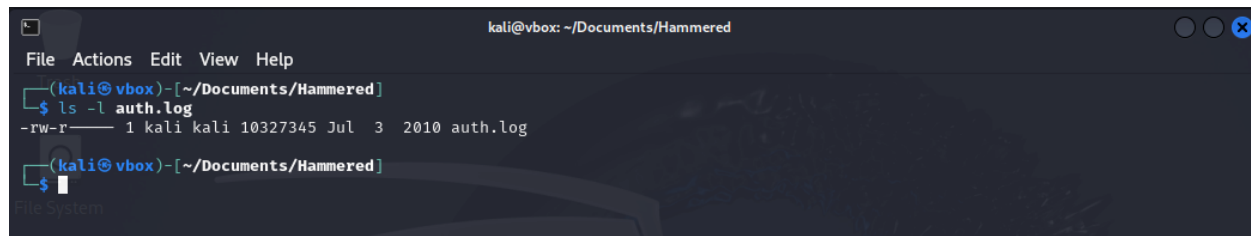
```
cat auth.log | grep -i "accepted password" | grep "219.150.161.20"
```

A terminal window titled 'kali@vbox: ~/Documents/Hammered' showing the command 'cat auth.log | grep -i "accepted password" | grep "219.150.161.20"' and its output. The output shows four lines of log entries from April 19, 2019, at 05:41:44, 05:42:27, 05:55:20, and 05:56:05, all showing successful password acceptance for root from the IP 219.150.161.20 on various ports (51249, 40877, 55545, 36585) using ssh2.

```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep -i "accepted password" | grep "219.150.161.20"
Apr 19 05:41:44 app-1 sshd[8810]: Accepted password for root from 219.150.161.20 port 51249 ssh2
Apr 19 05:42:27 app-1 sshd[9031]: Accepted password for root from 219.150.161.20 port 40877 ssh2
Apr 19 05:55:20 app-1 sshd[12996]: Accepted password for root from 219.150.161.20 port 55545 ssh2
Apr 19 05:56:05 app-1 sshd[13218]: Accepted password for root from 219.150.161.20 port 36585 ssh2
(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 14: Entries in auth.log from the attacker

The output does not contain the year the entries were made. Can simply use the command `ls -l` to list the details of the file.



```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)~[~/Documents/Hammered]
$ ls -l auth.log
-rw-r----- 1 kali kali 10327345 Jul  3 2010 auth.log
(kali@vbox)~[~/Documents/Hammered]
$
```

Figure 15: Details of the auth.log file

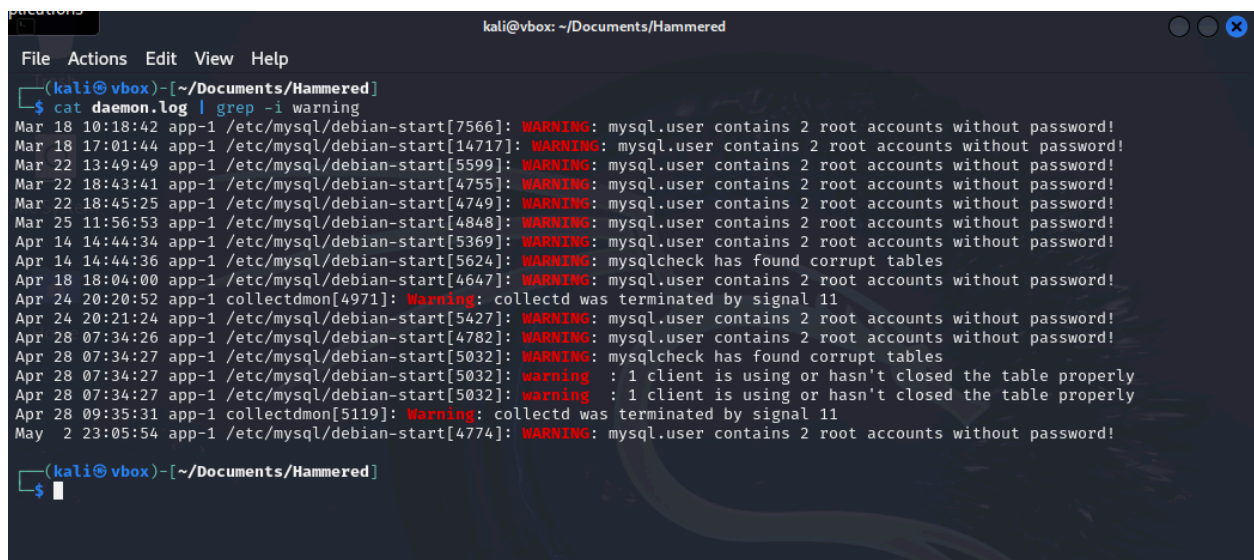
The file was created in the year 2010, therefore we can infer that the login entry was made on that year.

The attacker with IP 219.150.161.20 last login on 04/19/2010 05:56:05 AM

Q10: The database displayed two warning messages, provide the most important and dangerous one.

The daemon.log file records system service and background process events. If we output any lines from that file containing the word 'warning', can then determine which warning message is the most dangerous. Using the following command.

`cat daemon.log | grep -i warning`



```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)~[~/Documents/Hammered]
$ cat daemon.log | grep -i warning
Mar 18 10:18:42 app-1 /etc/mysql/debian-start[7566]: WARNING: mysql.user contains 2 root accounts without password!
Mar 18 17:01:44 app-1 /etc/mysql/debian-start[14717]: WARNING: mysql.user contains 2 root accounts without password!
Mar 22 13:49:49 app-1 /etc/mysql/debian-start[5599]: WARNING: mysql.user contains 2 root accounts without password!
Mar 22 18:43:41 app-1 /etc/mysql/debian-start[4755]: WARNING: mysql.user contains 2 root accounts without password!
Mar 22 18:45:25 app-1 /etc/mysql/debian-start[4749]: WARNING: mysql.user contains 2 root accounts without password!
Mar 25 11:56:53 app-1 /etc/mysql/debian-start[4848]: WARNING: mysql.user contains 2 root accounts without password!
Apr 14 14:44:34 app-1 /etc/mysql/debian-start[5369]: WARNING: mysql.user contains 2 root accounts without password!
Apr 14 14:44:36 app-1 /etc/mysql/debian-start[5624]: WARNING: mysqlcheck has found corrupt tables
Apr 18 18:04:00 app-1 /etc/mysql/debian-start[4647]: WARNING: mysql.user contains 2 root accounts without password!
Apr 24 20:20:52 app-1 collectdmon[4971]: Warning: collectd was terminated by signal 11
Apr 24 20:21:24 app-1 /etc/mysql/debian-start[5427]: WARNING: mysql.user contains 2 root accounts without password!
Apr 28 07:34:26 app-1 /etc/mysql/debian-start[4782]: WARNING: mysql.user contains 2 root accounts without password!
Apr 28 07:34:27 app-1 /etc/mysql/debian-start[5032]: WARNING: mysqlcheck has found corrupt tables
Apr 28 07:34:27 app-1 /etc/mysql/debian-start[5032]: warning : 1 client is using or hasn't closed the table properly
Apr 28 07:34:27 app-1 /etc/mysql/debian-start[5032]: warning : 1 client is using or hasn't closed the table properly
Apr 28 09:35:31 app-1 collectdmon[5119]: Warning: collectd was terminated by signal 11
May  2 23:05:54 app-1 /etc/mysql/debian-start[4774]: WARNING: mysql.user contains 2 root accounts without password!
(kali@vbox)~[~/Documents/Hammered]
$
```

Figure 16: Entries containing warning from daemon.log

The output shows duplicate warnings. Filtering out the duplicates we have two warnings.

1: Warning: collected was terminated by signal 11

2: WARNING: mysql.user contains 2 root accounts without password!

We can infer that root accounts without passwords is the most dangerous warning.

Q11: Multiple accounts were created on the target system. Which one was created on Apr 26 04:43:15?

Querying the auth.log with the timestamp, we can discover which account was created then.

```
cat auth.log | grep 'Apr 26 04:43:15'
```



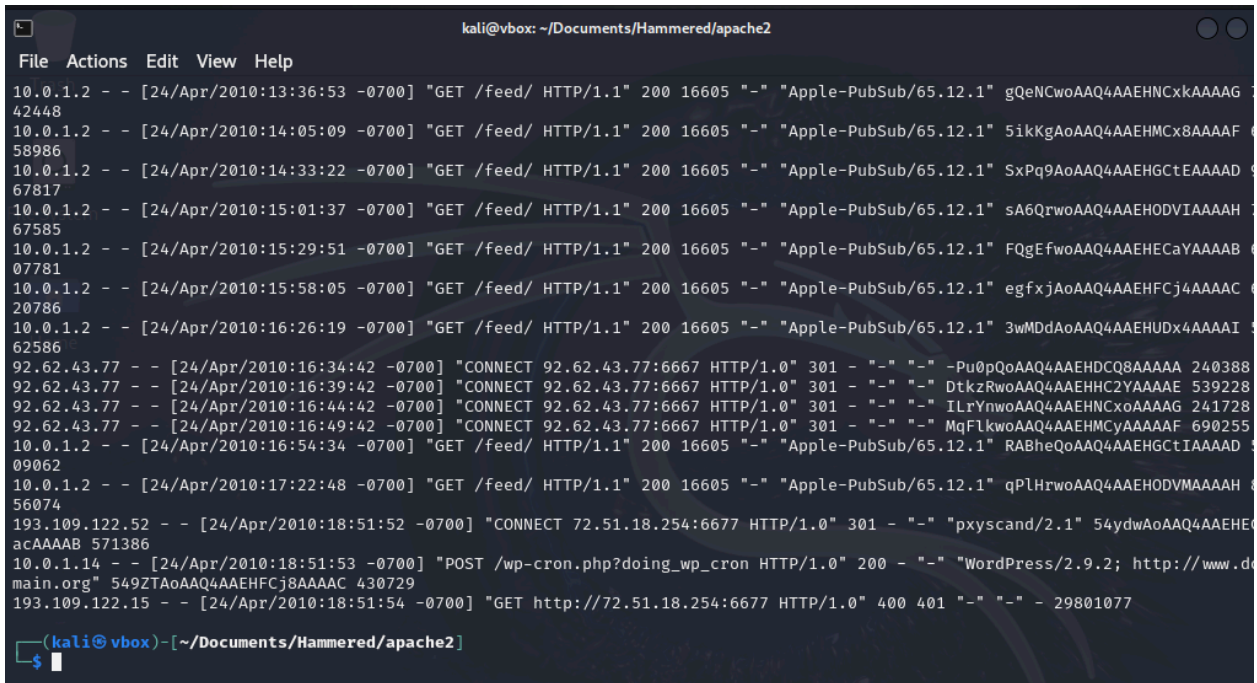
```
kali@vbox: ~/Documents/Hammered
File Actions Edit View Help
(kali@vbox)-[~/Documents/Hammered]
$ cat auth.log | grep "Apr 26 04:43:15"
Apr 26 04:43:15 app-1 groupadd[20114]: new group: name=wind3str0y, GID=1005
Apr 26 04:43:15 app-1 useradd[20115]: new user: name=wind3str0y, UID=1004, GID=1005, home=/home/wind3str0y, shell=/bin/bash
(kali@vbox)-[~/Documents/Hammered]
$
```

Figure 17: Shows account created on the given date

On April 26th 04:43:15, the account named 'wind3str0y' was created.

Q12: Few attackers were using proxy to run their scans. What is the corresponding user-agent used by this proxy?

For this question, we can look again at the file `www-access.log`. This file will contain user-agent information from incoming requests.



```
kali@vbox: ~/Documents/Hammered/apache2
File Actions Edit View Help
10.0.1.2 - - [24/Apr/2010:13:36:53 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" gQeNCwoAAQ4AAEHNCxkAAAAAG 42448
10.0.1.2 - - [24/Apr/2010:14:05:09 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" 5ikKgAoAAQ4AAEHMCx8AAAAAF 58986
10.0.1.2 - - [24/Apr/2010:14:33:22 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" SxPq9AoAAQ4AAEHGctEAAAAAD 67817
10.0.1.2 - - [24/Apr/2010:15:01:37 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" sA6QrwoAAQ4AAEHODVIAAAAH 67585
10.0.1.2 - - [24/Apr/2010:15:29:51 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" FQgEfwoAAQ4AAEHecAYAAAAAB 07781
10.0.1.2 - - [24/Apr/2010:15:58:05 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" egfxjAoAAQ4AAEHFCj4AAAAAC 20786
10.0.1.2 - - [24/Apr/2010:16:26:19 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" 3wMDdAoAAQ4AAEHUDx4AAAAI 62586
92.62.43.77 - - [24/Apr/2010:16:34:42 -0700] "CONNECT 92.62.43.77:6667 HTTP/1.0" 301 - "-" "-" -Pu0pQoAAQ4AAEHDC8AAAAA 240388
92.62.43.77 - - [24/Apr/2010:16:39:42 -0700] "CONNECT 92.62.43.77:6667 HTTP/1.0" 301 - "-" "-" DtkzRwoAAQ4AAEHHC2YAAAAE 539228
92.62.43.77 - - [24/Apr/2010:16:44:42 -0700] "CONNECT 92.62.43.77:6667 HTTP/1.0" 301 - "-" "-" ILrYnwoAAQ4AAEHNCxoAAAAAG 241728
92.62.43.77 - - [24/Apr/2010:16:49:42 -0700] "CONNECT 92.62.43.77:6667 HTTP/1.0" 301 - "-" "-" MqFlkwoAAQ4AAEHMCyAAAAAF 690255
10.0.1.2 - - [24/Apr/2010:16:54:34 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" RABheQoAAQ4AAEHGctIAAAAD 09062
10.0.1.2 - - [24/Apr/2010:17:22:48 -0700] "GET /feed/ HTTP/1.1" 200 16605 "-" "Apple-PubSub/65.12.1" qPlHrwoAAQ4AAEHODVMAAAAH 56074
193.109.122.52 - - [24/Apr/2010:18:51:52 -0700] "CONNECT 72.51.18.254:6677 HTTP/1.0" 301 - "-" "pxyscand/2.1" 54ydwAoAAQ4AAEHG 571386
10.0.1.14 - - [24/Apr/2010:18:51:53 -0700] "POST /wp-cron.php?doing_wp_cron HTTP/1.0" 200 - "-" "WordPress/2.9.2; http://www.d main.org" 549ZTAoAAQ4AAEHFCj8AAAAAC 430729
193.109.122.15 - - [24/Apr/2010:18:51:54 -0700] "GET http://72.51.18.254:6677 HTTP/1.0" 400 401 "-" "-" - 29801077
(kali@vbox) - [~/Documents/Hammered/apache2]
$
```

Figure 18: Output from `www-access.log`

Browsing through the output, one line is of interest. The line 5th from bottom that reads:

193.109.122.52 - - [24/Apr/2010:18:51:52 -0700] "CONNECT 72.51.18.254:6677 HTTP/1.0" 301 - "-" "pxyscand/2.1"

The string 'pxyscand' appears to be a program. Further investigation² pxyscand 2.1 is a network proxy scanner.

The scanning tool is pxyscand/2.1

² https://www.useragentstring.com/pxyscand2.1_id_17796.php

Conclusion

This lab involved investigation, inference, extensive use of Linux commands and working with log files to follow the trail of the attackers.

By delving into various logs I was able to improve my familiarity with the log formats and to identify which command would be best suited to retrieve the information required to answer the questions.

The investigation part of this lab was particularly interesting as it could represent a real world scenario. Trying to infer whether an attacker did attack the system by analyzing indicators of compromise from various logs.

Moving forward this lab solely focused on Detection phase in the incident response life cycle with relation to NIST SP 800-61 R3.

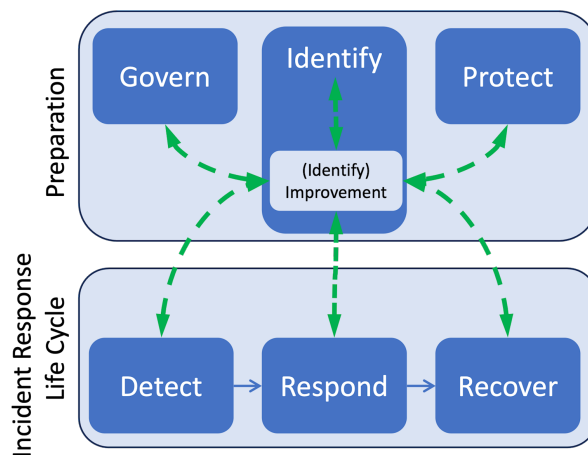


Figure 19: Incident Response Life Cycle³

Future improvements or to extend this lab could involve detailing the response and how the system would recover from such attacks.

³ <https://csrc.nist.gov/projects/incident-response>